CodeLLM Part 6: Scaffold Engineering and Optimization for a Gemma 4 SWE Agent (Plain-Language Edition)

Bo Hou (written up together with Claude) · 2026-10

Kaggle "Google – Gemma 4 Developer Agent", round one: local score on 129 tasks went from 51 to 58; first hidden-set submission scored 0.13 (leaderboard top at the time: 0.24)

Contents

0. What this note covers and how to read it
1. Starting point: what the competition wants and what we have
2. Step one: get the test set right
3. Baseline sub_v2: make it run, then measure
4. First iteration round under a 5-minute budget (v3–v8)
5. Taking the trajectories apart: where does the time go?
6. Bo's direction: the CPU searches and ranks, the LLM only judges
7. Overnight automated search: v9–v14
8. Can we trust the number? A health check of v9's 58/129
9. Packaging, time budgeting, and submitting to Kaggle
10. Notes on how we worked together
11. Where we are and what comes next
12. Appendices: version table, key numbers, glossary, links

0 What this note covers and how to read it

Part 5 covered "what this competition is, how SWE agents and RL could approach it, and what others are doing". Part 6 is the hands-on part: from standing up the environment to getting a first score on Kaggle. For each step it covers what we did, why, the pitfalls we hit, and which ideas the data shot down.

The one-sentence conclusion: **this round squeezed the scaffold (the prompts, tools and budgets around the model) about as far as it goes.** On the 129 local tasks we went from 51 to 58 solved, and cut the time per task from about 9 minutes on average to under 4. On the Kaggle hidden set, though, we scored only 0.13. Two main reasons: the model has seen many of the local tasks, and the hidden tasks are newer and harder. The next priority is to build a new task set that looks more like the hidden set, and then do RL on top of it.

Labels used in this note to show who decided what and who did what:

Label	Meaning
Bo's call	Direction, rules and trade-offs that Bo decided.
Claude's optimization	Analysis, design and implementation by Claude, followed by the "why".
Chrome executed	Hands-on work done by Claude for Chrome on the GPU machine: editing files, running experiments, reporting results.
Claude's mistake	Places where Claude judged or wrote something wrong, stated plainly.
Lesson	Things worth remembering in hindsight.

Division of labor: Bo works in VS Code and the browser, makes the decisions and does the review. The Claude in this conversation analyzes data, designs the approach and writes the instructions for Chrome. Claude for Chrome executes on the Lambda GPU machine: editing submissions, running evaluations, writing logs.

1 Starting point: what the competition wants and what we have

1.1 The goal in one sentence

Using the organizer-mandated `gemma-4-31b-it-qat-w4a16-ct` (31B, 4-bit quantized, the only allowed base model), build an agent that fixes bugs on its own. It reads a task (usually a GitHub issue or PR description), finds the problem in a Python repository, edits the code and hands in a patch. Grading runs hidden tests against that patch; if they pass, the task counts as solved. The metric is the fraction solved (Resolution Rate).

What you submit is not Python code but a "declarative" agent configuration: `agent.yaml` (which tools and which prompt to use), prompt files, sampling parameters, budgets, and optionally skill scripts and a LoRA. The organizer's compiler turns this into a Google ADK agent and runs it.

1.2 How evaluation runs: two containers

Phase	What happens
Container A (agent)	The agent uses tools inside `/workspace` to read code, run commands and edit code. At the end, the harness takes the changes as a patch with `git diff`.
Container B (grading)	Apply the agent's patch, reset any test files it touched back to the original, apply the hidden test_patch, then run pytest. The task counts as solved only if every required test passes.

Nine tools are available: `run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch`, plus three code-graph tools. Context is at most 32K tokens, with automatic compaction (older turns get summarized away).

1.3 What a task looks like: fastapi_11194 as an example

① **The task the agent receives** is a PR description. Simplified:

> 🐛 Fix: when a `File` parameter is declared after a `Form` parameter, the endpoint returns a 422 validation error. Right now `File` has to be declared first, or the request fails.

Along with the task, the agent gets its budget (minutes, number of calls), environment rules (offline, at most 150 lines per file read) and the repository's directory tree. The repository sits at `/workspace`, at the commit just before this PR was merged.

② **What the agent has to do**: find the FastAPI code that parses form parameters, fix the "breaks when the parameter order changes" problem, and hand in a patch. Note that `File` here is not a file name. It is a FastAPI parameter type (`File()`, whose parent class is `Form()`), so the agent has to search the code for that name. Here is how v7 went about it. It wrote a small script in /tmp to try to reproduce the bug. Then it ran `grep "class Form"` and `grep "class File"` and found `fastapi/params.py`. Reading it showed these were only definitions, not the bug. It then looked for where form parameters actually get parsed, and finally made the fix in `fastapi/dependencies/utils.py`. Just going from a name to its definition, and from the definition to the code that actually uses it, took several rounds.

③ **What happens at grading time**: the harness applies the agent's patch, then the hidden tests that came with the PR, and runs pytest. The hidden test looks roughly like this:

```
@app.post("/upload")
def upload(name: str = Form(), file: UploadFile = File()): ...

resp = client.post("/upload", data=..., files=...)
assert resp.status_code == 200     # unfixed: assert 422 == 200
```

The agent never sees the hidden tests. It can only guess from the task text what they will check. So some tasks fail on wording or details, for example when a test expects an exact error message that the task text never states.

1.4 Hardware and cost

	Local (ours)	**Kaggle grading machine**
GPU	Lambda A100-40GB, single GPU, $1.99/hour	4 × L4 (TP=4)
Inference	vLLM, 32K context, 3 workers running 3 tasks at once	vLLM, tasks run one at a time
Time limit	Our choice	At most 12 hours for the whole submission (about 120 hidden tasks)

Up to the first submission, Lambda cost about $280 in total. The machine is billed for every hour it is on, even with the GPU idle. So from here on: terminate the machine when nothing is running, and move analysis and debugging to a CPU machine.

1.5 Ground rules

> **Bo's call**
>
> 1. **The test set comes first and must be right.** If grading is wrong, every later comparison is meaningless.
> 2. **Squeeze the scaffold dry before thinking about RL.** In his words: "No rush. Look at the scaffold first, and only once it can't improve any further do we talk. I also want to analyze and look at things myself."
> 3. **Don't spend LLM reasoning on anything the CPU can do.** In his words: "If a program running on the CPU can go find the bug in the code repository, don't waste the LLM's reasoning on it."
> 4. **Every step must be understandable to a human.** Plain language, the right level of abstraction, no getting lost.

2 Step one: get the test set right

2.1 Why this comes first

The organizers released 129 public tasks across 4 repositories: fastapi, rich, requests and httpx. We grade them ourselves with a local replica of the harness and docker sandboxes. If the environment itself is broken (a missing dependency, a wrong version, something that needs the network), even a correct patch gets marked wrong. Comparing versions on such an environment is like measuring with a bent ruler.

2.2 Method: test the environment with the reference answer (gold patch)

The idea is simple: feed each task's reference patch to the grader as if it were the agent's output. If even the reference answer fails, the problem is the environment, not the agent. In the first pass, 108 of 129 tasks passed and **21** failed even with the reference answer.

2.3 The 21 ungradeable tasks, fixed by cause

Cause	Tasks	Symptom	Fix
anyio plugin	3	The grading command hard-codes `-p no:anyio`, so these fastapi async tests could not run	Added an environment-variable switch to the harness that changes it to `-p anyio` for these 3 tasks only; default behavior unchanged
Needs httpbin	8	The requests tests start a local HTTP server (pytest-httpbin), which the sandbox lacked	Built an image with httpbin and a matching werkzeug version. Also created a "tarpit" network: the container can reach services on itself but cannot get out to the internet, and the host blocks it with iptables
pygments version	7	rich's tests expect highlighting output that depends on the pygments version	For each task, searched for a pygments version under which the reference answer passes, and pinned it (2.13.0, 2.16.1, 2.19.2)
Python version	1	rich_3472 behaves differently on Python 3.13	Switched to a 3.12 image
Harness counts skip as fail	2	rich_3486 and fastapi_14186 have tests skipped by Python version, and the harness counted "skipped" as "failed"	Added env var `SWG_SKIP_AS_PASS`, applied only to these 2 tasks
Polluted dependencies	—	fastapi_14356 failed (see the mistake in 2.4)	Cleaned the wheel directory back to the organizers' original set
Time-dependent	1	requests_6757's test certificate has an expiry date and has expired by now	In this task's grading container only, used faketime to turn the clock back 250 days (separately authorized by Bo, for this task only)

After the fixes: **all 129/129 tasks are gradeable**. 108 pass under the default environment, and the other 21 get per-task environments through `overrides.json`; the regression check passed 22/22. Every change is controlled by an environment variable and default behavior is unchanged, so none of it affects other tasks.

> **Bo's call** Bo had Chrome do the environment fixes on its own, in his words "until the remaining failures are fixed". On rich_3486, Bo pointed out it was a harness bug: "It's a harness bug, it doesn't affect our use, and one environment variable is no big deal." So we worked around it with an environment variable rather than treating it as a defect in the task.

2.4 Claude's mistakes in this stage

Claude's mistake

1. **Wrong cause for fastapi_14356.** Claude first blamed the starlette version. The real cause turned out to be a bug in Claude's own environment-fix script: a misused `tr` command installed 45 extra wheels, and one of them, brotlicffi, changed how responses were compressed. Removing the extra wheels fixed the task.
2. **Wrong scope on the requests tasks.** Claude said "the timeout-related tests don't have to pass". In fact the harness requires the entire test file to pass.
3. **Wrong diagnosis on rich_3486.** Claude called it "a defect in the task itself". Bo pushed back: "Doesn't it depend on the Python version? Does pytest require both to pass?" That is what uncovered the real issue: the harness was counting skips as failures.
4. **Nearly overwrote existing settings.** The second version of a fix script (fix20b), if run as-is, would have wiped out the pygments versions already pinned. The third version merges instead of overwriting. Separately, the file-matching glob was case-sensitive and missed capitalized names like `Pygments-2.16.1`; it was changed to `[Pp]ygments`.

Lesson

1. "Does the reference answer pass?" is the cheapest and most reliable check of a grading environment. Do it first.
2. Scope every fix to the tasks it is for and leave default behavior alone, so fixing one task cannot break another.
3. Don't take an AI's diagnosis at face value: look at the evidence and have someone review it. Three of the four mistakes here were caught by Bo's follow-up questions.

3 Baseline sub_v2: make it run, then measure

3.1 Pitfalls before it ran end to end

Symptom	Cause	Fix
Conversations blew past 32K context after a few turns	The local CLI does not turn on the event compaction that Kaggle enables by default	Wrote a wrapper, `swegemma_local.py`, that turns on compaction with Kaggle's settings (compact every 5 events, threshold 14,336 tokens)
Import error: typing_inspection missing	The offline wheel directory lacked this package	Added the wheel and cleared the cache
fastapi tasks were "too easy"	The wheel directory contained a newer fastapi package, which effectively put the answer into the environment	Removed it; each repository's wheel directory now excludes the repository itself
One task called submit 470 times	The harness only ends after the agent replies with plain text, and the model kept submitting after handing in its patch	Added a "say one sentence and stop after submitting" rule to the prompt (the Stop clause), plus a turn limit
A whole night of results was lost	The venv used by the batch script did not have the kaggle package, so it failed on startup	Installed kaggle into the venv

3.2 Result: 51/129 (60 minutes and 100 calls per task)

sub_v2's setup: a main agent plus a "scout" sub-agent (investigator), with the Stop clause in the prompt. The 78 failures, by main cause:

Main cause	Tasks	In plain words
WRONG_BEHAVIOR:logic	20	Edited the right place but got the logic wrong
TOUCHED_TESTS	17	The agent edited test files, so the hidden test patch could not be applied and the whole task failed
EMPTY/BUDGET	14	Ran out of time or calls without changing a line
WRONG_FILES	11	Edited the wrong files
MISSING_NEW_FILE	6	The task needed a new file and the agent did not create it
WRONG_BEHAVIOR:message	6	The error message text did not match what the test expects
REGRESSION	4	Fixed this problem but broke something else

3.3 By year: a worry that later proved right

Grouped by the year of each task's PR: 2026 tasks pass at about 30%, earlier tasks at about 48% ($p \approx 0.06$). This suggests the model likely saw the older tasks during training. Bo's words at the time: "But the problem is the model Gemma gave us has seen all of these, and the tasks were made from…". Sections 8 and 9 show this worry was justified.

4 First iteration round under a 5-minute budget (v3–v8)

4.1 Why squeeze to 5 minutes

Kaggle allows 12 hours for the whole submission, and the hidden set has about 120 tasks, so about 6 minutes per task on average, and that has to include overhead such as container startup. sub_v2 averaged 8.8 minutes per task, which would time out on Kaggle. It had to come down.

> **Bo's call** "5 minutes is great, that's a big improvement." In other words: 5 minutes per task is a hard constraint; get the score up within it.

4.2 How we iterated: a dev set and noise

Running all 129 tasks for every change is too slow. Chrome picked 43 tasks as a **dev set**: the 12 where sub_v2 edited test files, the 14 with empty patches or blown budgets, 10 where it made the wrong fix, plus 8 tasks sub_v2 solved, as a control group. Each version changes one thing, and we compare dev-set scores. The noise is about ±2 to 4 tasks; differences inside that range don't count.

4.3 What each version changed and how it did

Version	Change	Dev /43	Conclusion
v2	Reference: 60 minutes, 100 calls	8	The dev set was picked from its failures, so a low score is guaranteed; not directly comparable
v3	Budget only: 5 minutes, 60 calls	10	20 empty patches: the 5 minutes mostly went to the investigator sub-agent
v4	Added a "time budget" section to the prompt: start editing before half the time is gone	13	Empty patches fell from 20 to 7. The model almost never calls get_status to check the time
v5	Removed the investigator sub-agent	13	Same score, simpler structure; kept
v6	Protect test files: never touch tests/; check `git status` before submitting	17	Failures from touched test files went from 5 to 0, a real gain
v7	Tool-call limit from 60 to 30	17	Same score, median time per task down from 308 s to 221 s; kept
v8	**Must** run related regression tests after editing	16	No gain, and more empty patches: the tests ate the call budget. Rejected

Lesson

1. In 5 minutes, **every extra step crowds out time for actually editing code**. v8's lesson: verification must be optional, and it must be cheap.
2. The gains came mostly from "not making dumb mistakes", like leaving test files alone and not burning time on a sub-agent, not from the model getting smarter.
3. Bo asked: "v2 got 51/129, how did v3 through v8 only add 3?" The answer: the dev set was built from v2's failures, so the two numbers aren't comparable. The dev set is only for comparing v3 and later.

5 Taking the trajectories apart: where does the time go?

Claude's optimization After v8, iteration stalled. Claude then took apart all 132 trajectories from v6, v7 and v8; every step has a timestamp, tool, arguments and result. The findings below set the direction for everything after.

5.1 98% of the time is the model "thinking"

Running a tool takes a median 0.06 s; a model call takes a median 2.7 s. But model calls have a heavy tail: **calls over 20 seconds are only 11% of calls but eat 51% of model time**. Each version had about 60 thoughts over 3,000 characters. The worst case: the call

budget was already used up and the patch was empty, and the model thought for another 13,485 characters (about 100 seconds) only to conclude "I'm done".

Bo asked at the time: does this include time running programs? If that's the CPU, can the GPU be freed up? Answer: no. CPU time for tools adds up to about 3 seconds per task; the bottleneck is entirely the GPU generating text.

5.2 29% of code-edit tool calls are broken

`edit_file` takes two pieces of code: the original snippet (old_string) and the replacement (new_string). Of 393 calls, **112 failed with "old_string missing"**. Here is why. When the model calls a tool, each argument is written as `name:<|"|>value<|"|>`, where `<|"|>` is a dedicated quote token. The model writes new_string first (alphabetical order), but ends it with an ordinary backtick `` ` `` instead, so the parser swallows the following old_string into new_string:

```
new_string:<|"|>self.external_docs = external_docs` ,old_string:<|"|>old code<|"|>
                                    ↑ should be <|"|>, written as a backtick
```

`run_command`, which takes a single string, was called 1,663 times and never broke once. The broken calls cluster in a few tasks where the model keeps retrying: fastapi_13713 alone retried 21 times.

5.3 File reads only reach the top of the file

25% of `read_file` calls passed no line numbers. The harness returns at most 150 lines per call, and without line numbers you only get the top of the file. The model gets the imports and header, doesn't find what it wants, reads again, and gets the top again. The logs even show the model saying "I keep reading the beginning of the file".

Bo asked: "read_file has line-number parameters, so why doesn't it use them? Is it Gemma that doesn't add them?" Yes. `agent.yaml` only decides which tools exist; the arguments of each call are the model's choice. It usually leaves out line numbers because it doesn't yet know which line the target is on.

5.4 30 calls run out before 5 minutes do

A "call" here means the agent calling one of the harness's tools. Every time the model decides to run a command, read a file or edit some code, that's one call. 30 is the limit we set for v7 in `eval_config.yaml` (`max_tool_calls`). The harness counts it and ends the task when it's used up. `submit_patch` and `get_status` don't count.

In v7, 5 tasks used up all 30 calls at about 160 seconds without changing a line and handed in empty patches, with more than 2 minutes still left. About 80% of calls went to grep and file reads, and the median time to the first edit was 130 to 150 seconds.

5.5 Other problems

- **Repeated calls**: the same command was run up to 18 times.

- **Scratch files in the patch**: debug scripts left in the repository got submitted along with the fix.
- **Some tasks can't be won by process changes**: fastapi_14479's test matches an exact error message the task never gives; fastapi_14953 needs a new file the task never mentions.

5.6 The code-graph tools: what they are and why they went unused this round

The "code graph" is a "who relates to whom" graph that the organizers prebuilt for each repository (NetworkX format):

- **Nodes**: every function, class and module in the repository.
- **Edges**: who calls whom (CALLS), which module something is defined in (DEFINED_IN), who imports whom (IMPORTS).
- Each node also carries a 256-dimensional embedding for finding code that "looks similar".

Tool	What it does
`get_code_neighbors(name)`	Finds upstream and downstream: who calls this function, and what it calls
`search_similar_code(name)`	Finds similar functions via embeddings. Takes a symbol name only, not a sentence
`get_code_subgraph([names…])`	Returns the relations among several functions at once

These tools are optional: we choose which tools to include, and v9 left them out. **The theoretical benefit** is walking the call chain in one call instead of grepping layer by layer. That could help on "wrong file" tasks, where the place that really needs fixing is often one level up or down the call chain.

What actually happened: across 46 tasks the model used them only 1 to 3 times in total, and one call came back empty. When v14 added them back, they were never called and the score didn't move. What they do is largely covered by `swe refs`, `swe def` and `swe locate` (see section 6), which read the current code.

> **Bo's judgment** On complex tasks, or when the agent can't find or picks the wrong spot, the code graph may well be useful. This round's data only says "the model won't use it on its own", not "it's useless". So next round it becomes a fallback (see section 11).

6 Bo's direction: the CPU searches and ranks, the LLM only judges

6.1 In his words

Bo's call

> "When you write it, you can generate some API functions for Gemma to call when needed. Take a subroutine like grep: lots of the work is repetitive, and whatever can be abstracted out for the CPU to do (especially sorting and searching) should go to the CPU. The formal sorting and searching inside a scaffold agent shouldn't use the LLM's reasoning... Try to get these 129 cases solved better. Gemma's own ability is limited; you're strong, so give it interface functions and knowledge and get the scaffold right."

> "Careful: the scaffold tricks must not overfit these 129 tasks, because in the end it's a hidden test set. It has to generalize."

And one key judgment: "Searching for the strongest scaffold should be left to AI, the way OpenAI searches for proofs of open math problems. Afterwards I just need to understand why the strongest scaffold is strong."

Why this direction is right: section 5 showed that time goes to "model thinking plus round trips", not the CPU. A grep takes 0.06 s, but every grep costs a round of model thinking plus a call, and one round is 3 to 8 seconds. **The key to saving time is cutting the number of rounds**: one CPU call should do the work that used to take seven or eight.

6.2 What swe is: not a harness tool, but a command we brought along

The harness tools are the 9 the organizers hard-coded; we can't add new ones. `swe` is a **skill script** shipped in the submission, just an ordinary Python file, `skills/swe/scripts/swe.py`. The competition allows skill scripts in the submission, and they run inside the agent's container, where they can see the repository. It plugs in like this:

1. The first time the agent calls `run_skill_script` (a tool that comes with the skill mechanism), the script installs itself as a command, `swe`, inside the container;
2. After that the agent runs `swe locate ...`, `swe edit ...` with the harness's existing `run_command`, just like `grep` or `ls`.

So the harness is untouched. We simply used its existing "run a command" tool and added a better command inside the container. From the model's point of view, it just has one more very capable command-line tool. This amounts to **bypassing the organizers' code graph with a more direct approach**: instead of a prebuilt graph, scan the current code on the spot. The two aren't fully equivalent: the code graph can follow call relations several levels up and down, while `swe refs` only goes one level.

6.3 The swe toolbox: a CPU interface for Gemma

Claude's optimization Claude designed a set of commands, and **Chrome executed** Chrome implemented them using only the Python standard library as `swe.py`. Unit tests ran in sandbox images for 4 Python versions (3.10 to 3.13): 47/47 passed. Each command maps to a problem from section 5:

Command	What it does	Problem it addresses
`locate words…`	Takes a few keywords from the task and returns the 10 most likely places to edit: file, line range, function name, which words matched, and the most relevant line of code	Finding code takes 7–8 rounds (5.4)
`outline FILE`	Lists the line ranges of every class and function in a file, like a map	Reads only reach the top of the file (5.3)
`show FILE symbol/line`	Shows code by function name or line number, with line numbers, at most 120 lines	Reads only reach the top of the file (5.3)
`def` / `refs`	Finds where a name is defined anywhere in the repo, or who calls it, grouped by enclosing function	Grepping back and forth
`edit FILE`	Takes "find this text → replace with that" blocks via a heredoc. If the original text isn't found, it shows the closest match; after editing it checks syntax and rolls back automatically on errors	29% of edit_file calls broken (5.2)
`insert` / `replace` / `new`	Insert by line, replace by line, create a new file; same syntax check and rollback; refuses to write test files	Tasks that need a new file
`tests` / `test`	Finds the relevant tests; runs them with a time limit and prints only a condensed failure summary	Verification must be cheap (lesson from 4.3)
`check`	One-stop pre-submit check: restore touched test files, delete scratch files, check syntax, show the diff, warn on an empty patch	Touched test files, scratch files in the patch (5.5)
End of every output	`[swe #n t=seconds]`: shows time used; past the limit, it says to run check and submit	The model doesn't know the time (v4's finding)
Repeat brake	A call identical in arguments, input and repo state gets a warning at the top: "same as call k, try something else"	Repeated calls (5.5)

Why edits use "find this text → replace with that" blocks:

```
swe edit pkg/mod.py <<'EOF'
<<<<<<< SEARCH
original code (a few lines, must be unique)
=======
new code
>>>>>>> REPLACE
EOF
```

- The whole block travels as one string argument through `run_command`, the path that never broke in 1,663 calls. That sidesteps the problem in 5.2.
- Many models have seen this format before. The delimiter is quoted, `'EOF'`, so the shell leaves `$` and backticks in the code alone.
- Matching has three tiers: exact, then ignoring trailing whitespace, then ignoring indentation (with automatic re-indentation). If there are several matches, it reports each one's line number and asks for more context.

6.4 Three fixes for "30 calls run out first"

#	Fix	Why it helps
①	`swe locate` does the grep and the ranking in one go, returning the most likely files, line ranges and functions	A location that used to take several greps and file reads now comes back in a single call
②	`swe outline` lists a file's functions and line numbers; `swe show` shows code by function name or line number	No more "read the top, miss the target, read again"
③	Call limit raised from 30 to 40	Each call does more work now, and time is the real hard limit; leave some headroom

Effect: median time to the first edit dropped from 131 s to 100 s, and fewer tasks ran out of calls and handed in empty patches.

6.5 How locate ranks (in plain words)

Put simply, locate is **a small search engine just for code**, which is exactly Bo's "let the CPU do the ranking and searching". It works much like BM25 ranks web pages:

1. Python's built-in ast splits all non-test code in the repository into individual functions and classes.
2. Each keyword is weighted by how rare it is (IDF): `self` and `return` appear everywhere and add almost nothing; `external_docs` appears in two or three places and adds a lot.
3. A function's score is the sum of the weights of the keywords it contains. Extra points if the function's own name matches, or if it contains a whole error message quoted in the task. Scores are normalized by function length so that big functions don't win just by having more text.
4. Directories outside the package (examples, docs) are down-weighted. The output is the top 10 functions plus one line listing the most relevant files.

It differs from a normal search engine in three ways. ① **Each function is a "document"**, so it returns "which function", not "which file contains the word". ② **The index is rebuilt on every call**: the repos are small, a scan takes under a second, and the search always sees the latest edited code. ③ **It matches literally and doesn't understand meaning**: a word from the task has to actually appear in the code to be found. If the task says

"upload a file" and the code says `UploadFile`, the model has to come up with the right word itself. Point ③ is where the next improvement goes (see section 11).

The output looks roughly like this (format illustration only):

```
fastapi/dependencies/utils.py:850-905  request_body_to_args  [File Form UploadFile]  if
isinstance(...
fastapi/params.py:700-786              File.__init__         [File Form]  ...
```

Offline check: on the 129 tasks, we extracted keywords from each task description automatically, ran locate, and checked whether the files changed by the reference answer showed up in the top 5. On the dev set, **file@5 = 0.84**: for 84% of tasks, the file that needs editing is in the top 5. This used only CPU, no GPU.

6.6 Where do skill scripts run? Experiment first, then design

Submissions may include a `skills/` directory and .py scripts, but the README doesn't say where scripts execute. If they ran on the host, they couldn't see the repository and the toolbox would be useless. So Claude had Chrome run a probe first: a minimal skill that prints its machine name, working directory, and whether it can see `/workspace`.

The answer was "A'": `run_skill_script` does run inside the agent container, but each time in a throwaway temporary directory. The workaround: the first time swe runs, it installs itself into `/usr/local/bin`, and from then on it's called as `swe ...` through `run_command`. Each `run_skill_script` call also counts as a tool call.

> **Lesson** When a mechanism isn't documented, ten minutes on a probe experiment beats getting the whole downstream design right by guessing. In case skills didn't work on Kaggle, the prompt also has a fallback: if skills are unavailable, do the same things with ordinary commands like grep and sed.

6.7 Key points of the prompt rewrite

- **Fixed a false statement.** v7's prompt said "if you don't call submit, everything is discarded". That's wrong: on timeout or when calls run out, the harness still grades the working-tree changes as the patch. The false statement made the model panic, so it became "your changes are graded no matter what; get an edit in first, then improve it".
- **A recommended workflow.** ① Make locate the first call, with 3 to 8 words from the task; ② use show, def, refs 1 to 3 times to read code; ③ edit with swe edit, with the first edit in place by about 120 seconds or the 12th call at the latest; ④ if there's time, do one cheap verification; ⑤ run check, submit, and end with one sentence.
- **General correctness knowledge.** The task is often a PR description; implement every item it lists. Copy names and error text from the task verbatim. Match the style of nearby code. Keep changes minimal and backward compatible. Export new public names in the same place their sibling modules do. If the task mentions examples, create the example files too.

- **Constraints.** No repository names or task IDs anywhere in the prompt, and no `{identifier}` braces: ADK treats them as variables to fill in and crashes if it can't find them.

6.8 Configuration

Parameter	v7	v9	Why
Tool list	9 native tools	`run_command`, `get_status`, `submit_patch` plus the skill	edit_file breaks 29% of the time, read_file without line numbers reads the top of the file, and nobody used the graph tools, so everything goes through swe
thinking_budget	4096	1024	Targets the long tail in 5.1
max_output_tokens	8192	6144	Shorter thinking needs less room
max_tool_calls	30	40	Each call is worth more now; time is the real hard limit
timeout_seconds	default 300	90	Keeps a single command (say, an accidental full test run) from eating the whole budget

7 Overnight automated search: v9–v14

7.1 Rules

Bo's call

- Chrome runs at maximum thinking effort and designs, implements, experiments and decides on its own, all night.
- **Guard against overfitting**: iterate only on the 43-task dev set; the remaining 86 tasks are held out, not to be looked at during iteration, and used only at the end to test generalization. No repository names, task IDs, or rules written for a specific task in any rule or script.
- Bo only looks at the final conclusions and "why it's strong".

Claude's instructions to Chrome laid out every fact, number and constraint from the earlier work, and required each version to report a set of mechanism metrics: time to first edit, empty-patch count, edit failure rate, repeated-call count, and usage of each swe command. A single total score was not enough.

7.2 Results

Version	Change	Dev /43	Notes
v7	Previous best	17	
v9	swe toolbox + rewritten prompt + the configuration in 6.8	**18**	Held-out 40/86, **full set 58/129**, 3.8 minutes per task on average
v10	Tool-side patches for "reads but never edits" loops	16	
v11	Thinking turned off	13	2.7× faster, but 3 to 5 fewer tasks solved
v12	Call limit back to 30	16	Saves no time: long tasks are cut off by the time limit, not the call count
v13	Three fixes to editing and checking	15	All fixes worked; the tasks lost were ones that flip between pass and fail anyway
v14	Added back 6 of v7's native tools	16	Half of the write_file calls were rejected; they were all attempts to write scratch files to /tmp. Not an OS permission issue: the harness's write_file only accepts paths inside /workspace. /tmp is writable, but through run_command plus a heredoc. v7's prompt said so, but the model kept its habit of using write_file and wasted calls. The graph tools were never called

Across 8 dev-set runs, the standard deviation of a single run's score is about 1.6 tasks, so two versions within about 2.2 tasks of each other can't be told apart. v9's 18 is on the lucky side.

7.3 The mechanics improved; the score barely moved

Metric	v7	v9
Broken edit calls (missing argument)	about 29%	0
Repeated calls (dev-set total)	58	36
Time to first edit (median)	131 s	100 s
Thoughts over 4,000 characters	40	2 (so the thinking cap does take effect)
Full set	est. 58 ± 5	58

Claude's judgment The efficiency problems are essentially solved, yet the score hasn't moved. So what's blocking us now is **the model misunderstanding and making wrong**

fixes, not efficiency. Further evidence: turning thinking off loses 3 to 5 tasks, so the score rests on the model's own reasoning. More rules on the scaffold will yield little.

In hindsight, nearly every optimization this round targeted **locating and operating**: find fast, read accurately, edit reliably, submit cleanly. There was almost no specific design for **getting the fix right** (understanding the task, writing correct logic, not missing requirements). Most remaining failures are in that layer, which is why efficiency metrics improved clearly while the score stayed put. The 43-task dev set still has 25 failures, more than half.

8 Can we trust the number? A health check of v9's 58/129

Bo asked: "Is there a way to check these results in ipython, and whether they're trustworthy?" Claude wrote a read-only check script. Results:

Check	Result	What it means
95% confidence interval	36.6% to 53.6%, about 47 to 69 tasks	58 should be read as a range, not a precise number
Patch identical to the reference line by line	**19 tasks**	Almost certainly the model memorized answers from these public repositories. Without them it's about 39/129 ≈ 30%
Tasks over 300 seconds	38, longest 405 s	The recorded time includes grading. The agent itself maxed out at 300.7 s; the time limit is strictly enforced
Union of tasks solved by any version	72/129	14 more than v9: tasks the model sometimes solves, the best candidates for RL
Scratch files in the patch	fastapi_5077 shipped module_a.py, module_b.py, repro.py, test_sig.py	That run was cut off at 287 s and never called check

Claude's mistake

1. The first version of the check script had two bugs. In quiet mode pytest's summary line has no `===`, so the regex missed it; and the reference patch format differed from what was assumed, so the "does the edited file overlap the reference" check reported no overlap for all 58 tasks. Bo questioned that result right away, and only then did Claude realize the script was at fault.
2. Claude took `fastapi/temp_pydantic_v1_params.py` for a scratch file the agent left behind. Chrome checked and found it already exists in the repository, and the reference answer edits it too.
3. Claude said fastapi_5077 "modified a test file". It was actually a new test_sig.py created at the repo root, not a change to an existing test file.

Lesson Check scripts need checking too. Whenever a result says "all 58 tasks are anomalous", suspect the script before the data.

9 Packaging, time budgeting, and submitting to Kaggle

9.1 How many minutes per task: the derivation

What Chrome found (from the competition pages and staff answers in the discussion forum): the hidden set has about 120 tasks, **run one at a time**; the 12 hours include container preparation but not grading; going over 12 hours fails the whole submission.

Component	Mean	Max
Container start + repo unpack + wheel install	3.5 s	6.5 s
Agent	205 s (under the 5-minute limit)	300.7 s
Patch extraction + container teardown	5.1 s	about 41 s
Per-task overhead on Kaggle (inferred from the official notebook)	13 to 16 s	assume 44 s

Worst case: 120 × (240 + 16) s ≈ 8.6 hours; with 44 s overhead on every task, 9.5 hours; add about 13 minutes for environment setup and model loading, for 8.8 to 9.7 hours total. With 5 minutes, the worst case is about 10.8 hours, which doesn't leave a 20% margin. So we chose **4 minutes**. The cost is losing up to about 8 of the 58 tasks we could otherwise solve.

9.2 Three submission versions

Version	Change	Verification
v9k	v9 with one change: 4-minute limit	Static checks (extensions, symlinks, braces, repo names, task IDs), the organizers' compiler, zip structure, smoke test 3/3 passed
v9k2	Better cleanup rules in check: delete new files at the repo root and files whose names contain repro, temp, tmp, etc. Plus a safety net: past the time threshold, any swe command also runs a cleanup	Offline check on 129 patches: 5 files would be deleted, and **none of the 49 files the reference answers create would be wrongly deleted**; replay on 5077's workspace; 17 new unit tests; live run 5/5 passed
v9k3	"About 5 minutes" in the prompt changed to 4; the TIME LOW warning moved from 200 s to 160 s, keeping the original proportion	4 of 5 tasks passed on the first run. The failing 5077 was rerun 4 times: v9k2 and v9k3 each passed 2 of 3 runs. It's a task that flips between pass and fail, not an effect of the change

> **Bo's call** Reference answers may be used to **verify** that the cleanup rules don't wrongly delete files, but they must never go into any submission. v9k3 was judged "random variation, OK to submit", and Bo made the call to submit.

9.3 Submission steps

1. Upload the zip as a Kaggle private dataset.
2. Copy the official Getting Started notebook and add the dataset with Add Input; select L4 × 4 GPUs and turn the internet off.
3. Change section 2 to use our files. **Be sure to delete the code that rewrites sampling.yaml**, or thinking silently goes back to 4096.
4. Keep the two-task local inference in section 5 to confirm the skill works in the Kaggle environment and to see real timings.
5. Save & Run All, then Submit Prediction. One submission per day; scoring is queued.

9.4 Result: 0.13 (leaderboard top at the time: 0.24)

Local 0.45 became 0.13 on the hidden set, roughly 3/10 of it. Someone else publicly reported about 1/5 (34.5% local, 6.9% hidden). Three reasons for the drop:

1. **Memorized answers.** 19 local patches are line-for-line identical to the reference. Without them, the local "real skill" score is about 30%.
2. **Hidden tasks are newer and harder.** Local 2026 tasks already pass at only about 30%, and the hidden set is likely tasks the model has never seen.
3. **Slower hardware, shorter time limit.** With 4 × L4 and 4 minutes, the agent gets fewer steps in the same time than it does locally.

Conclusion: **these 129 local tasks no longer measure hidden-set ability accurately.** A few more local solves would likely make no visible difference on Kaggle. For a first version, 0.13 against a leader at 0.24 is a reasonable starting point.

10 Notes on how we worked together

1. **Three-way division of labor.** Bo sets direction, decides and reviews; the Claude in this conversation analyzes, designs and writes instructions as ready-to-paste prompts; Chrome executes on the machine and brings back written reports. The biggest benefit is that Bo doesn't have to babysit the machine. In one night Chrome ran 6 versions plus a full-set run and the held-out set.
2. **Bo's review caught the most errors.** The diagnosis of rich_3486, the check script's "all 58 anomalous", "why did v3 to v8 only add 3"… all came from Bo's follow-up questions. Claude and Chrome also corrected each other, as with the scratch-file misjudgment.
3. **Plain language.** Bo repeatedly asked for "talk like a person", "the right level of abstraction", "be concise". Every interim report followed "conclusion → evidence → what you need to decide".
4. **Hard constraints written into every instruction.** Don't stop vLLM; don't delete results; reference answers never go into a submission; no repo names or task IDs; faketime for one task only; test containers stay offline; Claude Code authorization is clicked by Bo himself. Every prompt to Chrome repeated them.
5. **Money.** The machine costs money whenever it's on, idle or not. From now on: shut it down when nothing runs, use a CPU machine for analysis and debugging (grading phase two only needs CPU), and batch jobs into overnight runs.
6. **Every task should be understandable to a human.** Bo wants to debug for himself why each task failed, so the next step prepares two tools for a CPU machine: `case_view.py` (one folder per task: the task, agent patch vs reference, failing tests, what the agent did at each step, a one-line failure reason) and `retest.sh` (re-grade any patch).

11 Where we are and what comes next

Status: v9k3 is submitted, scoring 0.13 on the hidden set. Locally it's 58/129, of which 19 look memorized. All versions together have solved 72. The scaffold is essentially squeezed dry on efficiency.

> **Bo's roadmap** "Later we can randomly pick some harder tasks (construct new tasks), let AI extend the scaffold until it can't improve any further, and then look at how to bring in RL to keep improving Gemma."

> **Bo's judgment** Gut feeling says the scaffold isn't squeezed dry yet: more than half of the dev set still fails, and all versions together solve 72 tasks, 14 more than v9. The next scaffold round works on two directions together: **locating** and **fixing correctly**.

11.1 Next scaffold round

Direction	What to do	Why
Read the failures first	Go through the 25 dev-set failures one by one, starting with the 14 "sometimes solved" tasks (fastapi_14266, fastapi_14616, httpx_3672, requests_7328, rich_3063, rich_3180, rich_3278, rich_3470, rich_3480, rich_3521, rich_3676, rich_3782, rich_4070, rich_4079). For each, decide: wrong place, or wrong fix?	The ratio between the two decides where the next round focuses
Locating	Make the code graph a **fallback**: write it into the prompt as a conditional rule (if locate's results are uncertain, or the repro still fails after the edit, use `get_code_neighbors` to look upstream and downstream). Or add `swe chain name` to swe, using ast to expand the call chain 2 to 3 levels up and down on the spot	Bo: needed for complex tasks or when the agent picks the wrong spot. Verify by comparing, on "wrong file" tasks, how many find the right location before and after
Locating	Make locate **hybrid lexical plus semantic retrieval**: today's literal matching plus semantic matching with the organizers' embeddings	Handles tasks whose wording differs from the names in the code
Fixing correctly	Reproduce first, then fix: write a small script in a scratch directory that reproduces the bug, and rerun it after the edit to confirm the fix	Avoids "thought it was fixed, but it wasn't"
Fixing correctly	Requirement checklist: before submitting, list every requirement in the task (new parameters, exact error text, new files, export locations) and tick them off one by one	Avoids missed requirements and mismatched error text
Operating	Create a scratch directory inside `/workspace` (e.g. `/workspace/.swe_tmp/`) and have swe add it to `.git/info/exclude` on first run, so it stays out of the patch	Bo's call. The model habitually writes scratch files in the repo; giving it a legitimate place works better than forbidding it

11.2 Longer term

#	What to do	Why
1	Build new tasks: from other Python repositories, pick PRs merged after Gemma's training cutoff, and verify each task's environment with its reference answer (the method from section 2, scripted for batch use)	Local scores need to represent the hidden set, with memorized points removed
2	Keep letting AI extend the scaffold on the new tasks until it stops improving	Gains on the 129 tasks don't show up on the hidden set
3	Find the "sometimes solved" tasks: sample each task 4 to 8 times and keep the ones with mixed outcomes	RL methods like GRPO can only learn from groups with both successes and failures
4	Freeze the scaffold and do RL or LoRA on top of it (needs a GPU with 80 GB or more)	So we can tell whether gains come from the model itself or from the scaffold
5	Compare versions task by task on the full set, or run the same version several times	Single-run dev-set noise is ±2 tasks, too much to tell versions apart

To be added: results of the CPU analysis (failure categories, the list of "sometimes solved" tasks, NEXT_STEPS.md), and the actual total runtime of this Kaggle submission.

Appendix A: Version table

Version	Budget	Key change	Dev /43	Full /129
v1	—	Main agent + investigator	—	—
v2	60 min / 100 calls	+ Stop clause, scratch files in /tmp	8	51
v3	5 min / 60 calls	Budget only	10	—
v4	same	Time budget in the prompt	13	—
v5	same	Investigator removed	13	—
v6	same	Test files protected	17	—
v7	5 min / 30 calls	Call limit 30	17	about 58 (est.)
v8	same	Mandatory regression tests	16	—
v9	5 min / 40 calls	swe toolbox, new prompt, thinking 1024	18	58
v10–v14	—	Assorted patches and ablations	13–16	—
v9k / k2 / k3	4 min / 40 calls	Kaggle build, cleanup rules, consistent time wording	—	Kaggle 0.13

Appendix B: Key numbers

Item	Number
Gradeable tasks	129/129 (108 on the default environment + 21 with per-task environments)
Share of time in the LLM	about 98%; calls over 20 s are 11% of calls and 51% of model time
Broken edit_file calls	112/393 (29%); run_command 0/1663
locate recall	file@5 = 0.84 (dev set)
Dev-set noise	single-run standard deviation about 1.6 tasks
v9 full set	58/129, 95% CI 47 to 69 tasks; 19 line-for-line identical to the reference; 72 solved by some version
Kaggle	about 120 tasks run one at a time, 12 hours; worst case about 8.8 to 9.7 hours at 4 minutes
Hidden set	0.13 (leaderboard top at the time: 0.24)

Appendix C: Glossary

Term	In plain words
scaffold	The "scaffolding" around the model: prompts, tools, budgets, workflow. Model weights stay unchanged.
harness	The organizers' evaluation program: starts containers, runs the agent, extracts the patch, grades it.
gold patch	The reference patch, i.e. the real change in the PR.
test_patch	The hidden tests applied at grading time.
dev set / held-out set	The dev set is for iterating and choosing versions; the held-out set is not looked at during iteration and is used at the end to check for overfitting.
skill	Scripts and instructions a submission can carry, such as the swe toolbox.
thinking budget	The maximum number of tokens the model may "think" before giving its answer on each turn.
GRPO	An RL method: sample the same task several times, reward the successful attempts and penalize the failed ones.

Appendix D: Links

What	Link
Competition page	https://www.kaggle.com/competitions/gemma-4-developer-agent
Google ADK (agent framework)	https://github.com/google/adk-python
ADK Skills docs	https://adk.dev/skills/
vLLM (inference server)	https://github.com/vllm-project/vllm
SWE-bench (where the task format comes from)	https://github.com/SWE-bench/SWE-bench
The 4 task repositories	https://github.com/fastapi/fastapi · https://github.com/Textualize/rich · https://github.com/psf/requests · https://github.com/encode/httpx
PR behind the example task fastapi_11194	https://github.com/fastapi/fastapi/pull/11194
Series blog	https://jsjhoubo.github.io